



南開大學  
Nankai University

# 强化学习实验 实验报告

实验名称

基于策略梯度算法的 Cartpole 问题求解

姓 名：韩炎旻  
学 号：2312225  
专 业：智能科学与技术

人工智能学院

2026 年 5 月

# 1 实验目标

本实验旨在使用策略梯度方法训练 Gym 中的 CartPole-v1 环境，完成从状态观测到动作选择的端到端策略学习。实验重点包括：

- 理解直接参数化策略  $\pi_\theta(a|s)$  的思想，与基于动作价值函数的 DQN 方法形成对比；
- 实现 REINFORCE 策略梯度算法，并使用神经网络近似随机策略；
- 观察折扣回报、回报标准化和批量更新对训练稳定性的影响；
- 通过训练日志和模型评估结果分析最终策略性能。

## 2 实验环境

### 2.1 CartPole-v1 环境介绍

CartPole 是强化学习中的经典控制问题。智能体需要控制小车左右移动，使竖直杆尽可能长时间保持平衡。该任务可以建模为马尔可夫决策过程：

- **状态空间**：连续状态向量，包含小车位置、小车速度、杆角度和杆角速度，共 4 维；
- **动作空间**：离散动作， $a \in \{0, 1\}$ ，分别表示向左或向右施加力；
- **奖励函数**：每保持一个时间步未失败，获得奖励 +1；
- **终止条件**：小车位置越界、杆角度过大或达到环境最大步数；
- **最高回报**：CartPole-v1 单回合最高回报为 500。

### 2.2 实验配置

本实验使用 Python、Gym/Gymnasium 与 PyTorch 实现。

## 3 策略梯度算法原理

### 3.1 策略参数化

与 DQN 直接学习动作价值函数  $Q(s, a)$  不同，策略梯度方法直接学习一个带参数的策略：

$$\pi_\theta(a|s) = P(a_t = a \mid s_t = s; \theta)$$

其中  $\theta$  为神经网络参数。对 CartPole 这种离散动作环境，策略网络输出每个动作对应的 logits，再通过分类分布采样动作。

### 3.2 目标函数

策略梯度的目标是最大化期望回报：

$$J(\theta) = \mathbb{E}_{\tau \sim \pi_\theta} [G(\tau)]$$

其中  $\tau$  表示由当前策略采样得到的轨迹， $G(\tau)$  表示轨迹回报。对某一时间步  $t$ ，折扣回报定义为：

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k r_{t+k}$$

REINFORCE 算法使用如下无偏梯度估计：

$$\nabla_\theta J(\theta) = \mathbb{E}_{\pi_\theta} [\nabla_\theta \log \pi_\theta(a_t|s_t) G_t]$$

由于 PyTorch 优化器默认最小化损失函数，代码中使用的损失为：

$$L(\theta) = -\frac{1}{N} \sum_{t=1}^N \log \pi_{\theta}(a_t|s_t) \hat{G}_t$$

其中  $\hat{G}_t$  为标准化后的折扣回报。

### 3.3 回报标准化

REINFORCE 使用完整回合回报进行更新，方差通常较大。实验中先对单回合折扣回报做标准化，再对整个 batch 的回报统一标准化：

$$\hat{G}_t = \frac{G_t - \mu_G}{\sigma_G + \epsilon}$$

该处理不会改变回报的相对大小，但可以减小梯度尺度波动，使训练更加稳定。

## 4 算法实现

### 4.1 策略网络结构

策略网络采用两层全连接隐藏层：

- Linear(4 → 128) + ReLU;
- Linear(128 → 128) + ReLU;
- Linear(128 → 2) 输出动作 logits。

网络没有显式添加 Softmax 层，而是在动作选择时使用：

`Categorical(logits=action_logits)`

这样由 PyTorch 在分类分布内部完成归一化，数值稳定性优于手动计算概率。

### 4.2 训练流程

实验采用批量版 REINFORCE，即每收集若干个完整 episode 后统一更新一次策略网络。算法流程如下：

---

**Algorithm 1** 批量 REINFORCE 策略梯度算法

---

- 1: 初始化策略网络  $\pi_{\theta}$  与 Adam 优化器
  - 2: **for** 每个 batch **do**
  - 3:   清空 batch 轨迹缓存
  - 4:   **for** 每个 episode **do**
  - 5:     重置环境，获得初始状态  $s_0$
  - 6:     **while** episode 未结束 **do**
  - 7:       根据  $\pi_{\theta}(a|s)$  采样动作  $a_t$
  - 8:       与环境交互，记录  $r_t$  和  $\log \pi_{\theta}(a_t|s_t)$
  - 9:     **end while**
  - 10:   从后向前计算折扣回报  $G_t$
  - 11:   将该 episode 的 log probability 和回报加入 batch
  - 12:   **end for**
  - 13:   标准化 batch 中的折扣回报
  - 14:   计算  $L(\theta) = -\log \pi_{\theta}(a_t|s_t) \hat{G}_t$  的平均值
  - 15:   反向传播并更新策略网络参数
  - 16: **end for**
  - 17: 保存训练后的模型
-

### 4.3 评估方式

训练阶段通过随机采样动作进行探索；评估阶段不再采样，而是选择策略网络 logits 最大的动作：

$$a_t = \arg \max_a \pi_{\theta}(a|s_t)$$

这样可以更稳定地测试训练后策略本身的表现。

## 5 训练结果与分析

### 5.1 训练过程

训练命令如下：

```
1 python .\policy_gradient_train.py --cpu
```

训练脚本会在训练结束后自动保存曲线图：

policy\_gradient\_training\_curve.png

训练共设置 1000 个 episode，batch size 为 10。训练日志显示，初期回合奖励多在 20 至 40 左右，说明随机策略难以长期保持平衡。随着策略更新进行，模型在中后期开始明显提升：

- episode 400 时，最近 50 回合平均回报约为 62.96；
- episode 640 时，最近 50 回合平均回报约为 259.10；
- episode 780 时，最近 50 回合平均回报约为 322.94；
- episode 880 时，最近 50 回合平均回报约为 356.26；
- episode 1000 时，最近 50 回合平均回报约为 288.48；
- 后期多次出现单回合 500 分，说明策略已经学会较稳定的平衡控制。

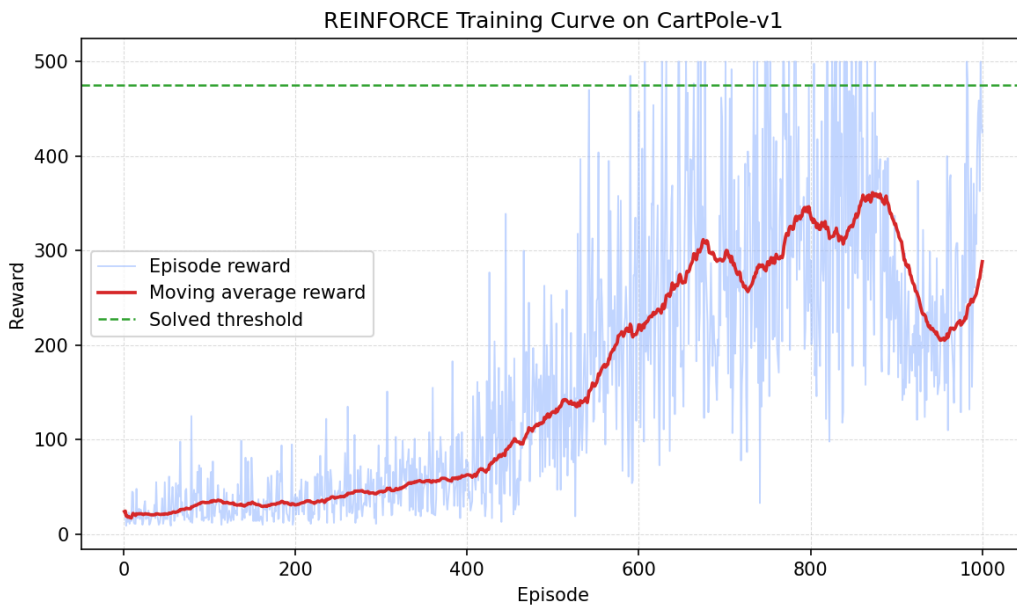


图 1: REINFORCE 在 CartPole-v1 上的训练曲线

图 1 中浅蓝色曲线表示单回合奖励，红色曲线表示最近若干回合滑动平均奖励，绿色虚线表示设定的求解阈值。可以看到训练前期回报较低且波动明显，中后期滑动平均奖励逐步上升，说明策略网络逐渐学习到有效的平衡控制策略。

训练结束后模型保存为：

policy\_gradient\_cartpole.pth

## 5.2 模型评估

对保存模型进行 5 回合评估，命令如下：

```
1 python .\policy_gradient_train.py --eval --eval-episodes 5 --model-path .\
    policy_gradient_cartpole.pth --cpu
```

评估结果见表 1。

表 1: 策略梯度模型评估结果

| 评估回合 | 回合奖励   |
|------|--------|
| 1    | 500    |
| 2    | 500    |
| 3    | 500    |
| 4    | 500    |
| 5    | 500    |
| 平均值  | 500.00 |

从评估结果可以看出，训练后的策略在 5 次评估中全部达到 CartPole-v1 的最高回报 500，平均回报为 500.00，说明模型已经掌握了较稳定的平衡控制策略。

## 5.3 结果分析

训练过程中最近 50 回合平均回报最终为 288.48，未达到代码中设置的严格求解阈值 475.0，但评估平均回报达到 500.00。造成这种差异的主要原因是训练阶段采用随机采样动作，仍保留策略分布带来的随机性；评估阶段采用贪心动作选择，因此表现更加稳定。

此外，REINFORCE 属于纯蒙特卡洛策略梯度方法，每次更新依赖完整轨迹回报，梯度估计方差较高，训练曲线存在明显波动是正常现象。通过 batch 更新和回报标准化，本实验已经在较简单的 CartPole 环境上取得了较好的训练效果。

## 6 超参数设置

表 2: 策略梯度训练超参数

| 参数                       | 数值                           |
|--------------------------|------------------------------|
| environment              | CartPole-v1                  |
| algorithm                | REINFORCE                    |
| episodes                 | 1000                         |
| batch size               | 10                           |
| discount factor $\gamma$ | 0.99                         |
| learning rate            | $1 \times 10^{-3}$           |
| hidden dimension         | 128                          |
| optimizer                | Adam                         |
| solved window            | 50                           |
| solved score             | 475.0                        |
| model path               | policy_gradient_cartpole.pth |
| device                   | CPU                          |

## 7 结论

本实验完成了基于策略梯度算法的 CartPole-v1 环境训练与评估。实验表明：

- REINFORCE 可以直接优化随机策略，不需要显式学习动作价值函数；
- 对折扣回报进行标准化能够改善梯度尺度，提升训练稳定性；

- 批量收集多个 episode 后统一更新，能够缓解单轨迹更新方差过大的问题；
- 最终模型在 5 回合评估中平均回报达到 500.00，具备稳定的平衡控制能力。

相比 DQN 这类基于价值函数的方法，策略梯度方法实现思路更加直接，天然适合随机策略和连续动作扩展；但基础 REINFORCE 方差较大、样本效率较低。后续可以进一步引入 baseline、Actor-Critic 或 PPO 等方法，以提升训练效率和稳定性。

## 8 附录：代码说明

实验代码位于 Github:

<https://github.com/Hemoritris/Reinforcement-Learning-Experiment-Notes/tree/main/Experiment%207/Code>